兼容STM32CubeMX 6.15 CmakeLists.txt DEMO

现在的选择！（未经过多测试）（未更新完）

直接复制以下内容到你的cmakelists.txt即可，一般不用修改cmakelists.txt, 除了你要加你的TASK文件夹

new_cmakelists

```
cmake_minimum_required(VERSION 3.22)

# Set the project name
set(CMAKE_PROJECT_NAME ACE_ECF_26projectF4)

# Setup compiler settings
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS ON)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIREdD ON)
set(CMAKE_CXX_EXTENSIONS ON)
set(SIZE arm-none-eabi-size)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# Define the build type
if(NOT CMAKE_BUILD_TYPE)
     set(CMAKE_BUILD_TYPE "Debug")
endif()

# Enable compile command to ease indexing with e.g. clangd
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# Add color diagnostics
set(CMAKE_COLOR_DIAGNOSTICS ON)

# Include toolchain file
include("cmake/gcc-arm-none-eabi.cmake")

# Core project settings
project(${CMAKE_PROJECT_NAME})

```

```cmake
# Enable CMake support for ASM, C and CXX languages
enable_language(C CXX ASM)

# Remove warning
add_compile_options(-w)

# Create an executable object type
add_executable(${CMAKE_PROJECT_NAME})

# Add STM32CubeMX generated sources
add_subdirectory(${CMAKE_SOURCE_DIR}/cmake/stm32cubemx)

# ==============================
# 包含dir以下后缀文件定义函数
# ==============================
function(USER_ADD_DIR dir)
    file(GLOB USER_SOURCE
            ${dir}/*.h
            ${dir}/*.c
            ${dir}/*.hpp
            ${dir}/*.cpp
            ${dir}/*.cc
    )
    target_sources(${CMAKE_PROJECT_NAME} PUBLIC ${USER_SOURCE})
    target_include_directories(${CMAKE_PROJECT_NAME} PUBLIC ${dir})
endfunction()
# ==============================
# 模块开关定义函数
# ==============================
function(ADD_MODULE name)
    # 定义模块开关（默认 OFF）
    option(USE_${name} "Enable ${name} module" OFF)
    # 取出除了第一个参数（模块名）之外的所有参数
    set(extra_paths ${ARGN})
    if(USE_${name})
        message(STATUS "✅ ${name} ENABLED")
        add_compile_definitions(USE_${name}=1)
        # 自动注册多个路径
        foreach(path IN LISTS extra_paths)
            USER_ADD_DIR(${path})
        endforeach()
    else()
        message(STATUS "❌ ${name} DISABLED")
    endif()
endfunction()

# ==============================
```

```
# 计入模块选择
# ============================
ADD_MODULE(SINGLE_USART
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/USART
)
if(EXISTS "STM32H723xx")
ADD_MODULE(SINGLE_CAN
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/DWT
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/CAN
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/CAN/fdcan
)
else()
ADD_MODULE(SINGLE_CAN
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/DWT
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/CAN
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/CAN/can
)
endif()
if(EXISTS "STM32H723xx")
    ADD_MODULE(DJI_MOTOR
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/DWT
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/App/safe_task
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/CAN
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/CAN/fdcan
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/module/motor
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/module/motor/DJI
    )
else()
    ADD_MODULE(DJI_MOTOR
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/DWT
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/App/safe_task
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/CAN
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/CAN/can
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/module/motor
            ${CMAKE_SOURCE_DIR}/ACE_ECF_26/module/motor/DJI
    )
endif()
ADD_MODULE(DR16
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/DWT
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/App/safe_task
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/USART
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/module/dr16
)
ADD_MODULE(REFEREE
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/DWT
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/App/safe_task
        ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/USART
```

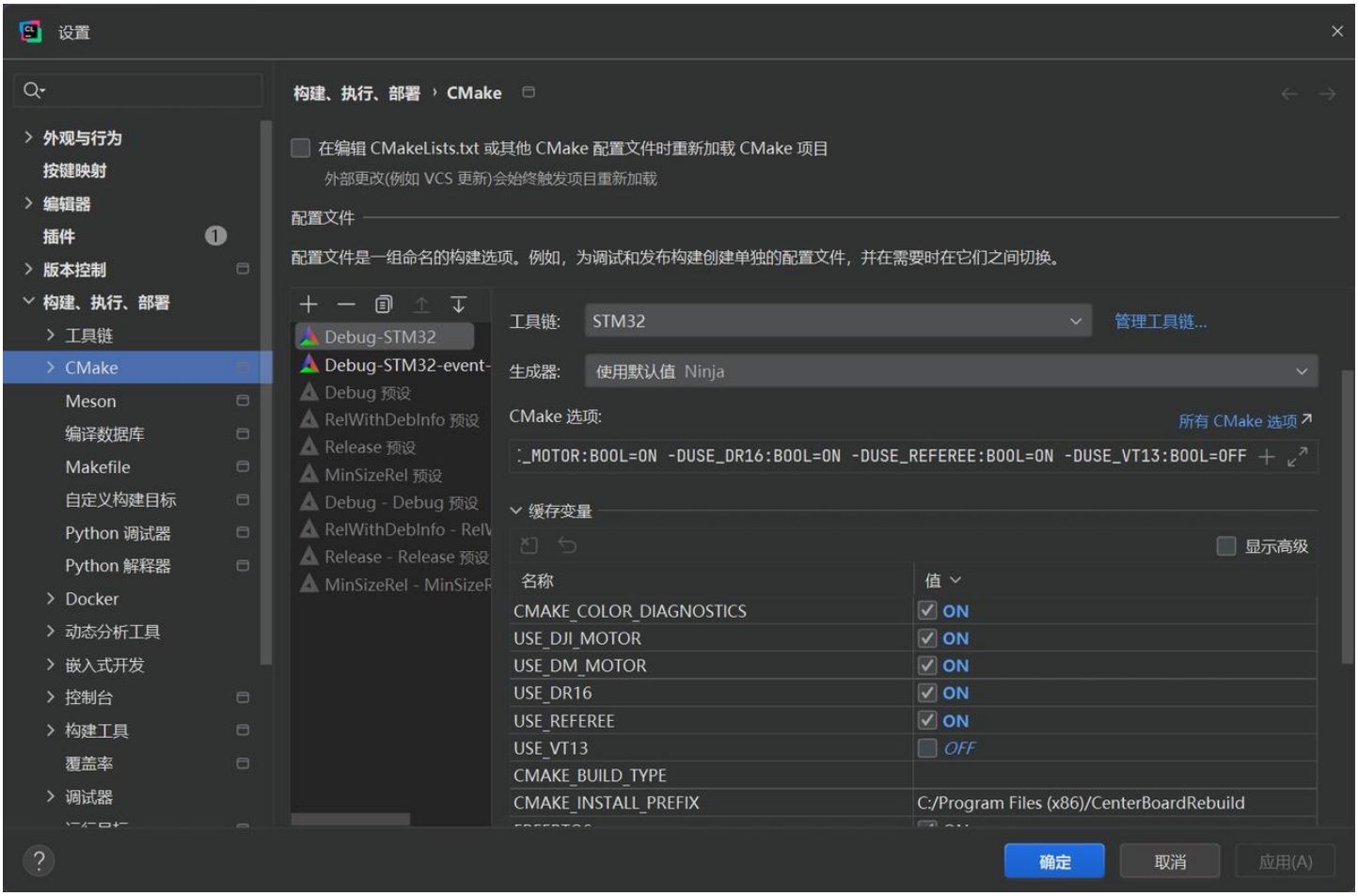
```
127         ${CMAKE_SOURCE_DIR}/ACE_ECF_26/module/referee
128 )
129 ADD_MODULE(VT13
130         ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/DWT
131         ${CMAKE_SOURCE_DIR}/ACE_ECF_26/App/safe_task
132         ${CMAKE_SOURCE_DIR}/ACE_ECF_26/Bsp/USART
133         ${CMAKE_SOURCE_DIR}/ACE_ECF_26/module/referee
134 )
135
136 # ==============================
137 # 26框架库固定文件
138 # ==============================
139 USER_ADD_DIR(${CMAKE_SOURCE_DIR}/ACE_ECF_26/App/robo_cmd)
140 USER_ADD_DIR(${CMAKE_SOURCE_DIR}/ACE_ECF_26/Algorithm/CRC)
141 USER_ADD_DIR(${CMAKE_SOURCE_DIR}/ACE_ECF_26/Algorithm/filter_alg)
142 USER_ADD_DIR(${CMAKE_SOURCE_DIR}/ACE_ECF_26/Algorithm/kalman_filter)
143 USER_ADD_DIR(${CMAKE_SOURCE_DIR}/ACE_ECF_26/Algorithm/lqr)
144 USER_ADD_DIR(${CMAKE_SOURCE_DIR}/ACE_ECF_26/Algorithm/PID)
145 USER_ADD_DIR(${CMAKE_SOURCE_DIR}/ACE_ECF_26/Algorithm/quaternion_kalman)
146 USER_ADD_DIR(${CMAKE_SOURCE_DIR}/ACE_ECF_26/Algorithm/user_maths)
147 USER_ADD_DIR(${CMAKE_SOURCE_DIR}/ACE_ECF_26/Algorithm/user_maths/cpp_files)
148 USER_ADD_DIR(${CMAKE_SOURCE_DIR}/ACE_ECF_26/Algorithm/user_maths/fifo)
149
150 # ==============================
151 # 自己的文件加这里（TASK）
152 # ==============================
153
154
155 # 判断芯片型号目录是否存在
156 if(EXISTS "${CMAKE_SOURCE_DIR}/STM32H723xx")
157     set(DSP_LIB_PATH
    "${CMAKE_SOURCE_DIR}/Middlewares/ST/ARM/DSP/Lib/libarm_cortexM7lfdp_math.a")
158 else()
159     set(DSP_LIB_PATH
    "${CMAKE_SOURCE_DIR}/Middlewares/ST/ARM/DSP/Lib/libarm_cortexM4lf_math.a")
160 endif()
161
162 # Add linked libraries
163 target_link_libraries(${CMAKE_PROJECT_NAME}
164         stm32cubemx
165         ${DSP_LIB_PATH}
166         # Add user defined libraries
167 )
168
169 add_custom_command(
170         TARGET ${PROJECT_NAME} POST_BUILD
```

```
171         COMMAND ${CMAKE_OBJCOPY} -Oihex $<TARGET_FILE:${PROJECT_NAME}>
    ${PROJECT_BINARY_DIR}/${PROJECT_NAME}.hex
172         COMMAND ${CMAKE_OBJCOPY} -Obinary $<TARGET_FILE:${PROJECT_NAME}>
    ${PROJECT_BINARY_DIR}/${PROJECT_NAME}.bin
173         COMMENT "Building hex & bin file...\nEXCUTABLE SIZE:"
174         COMMAND ${SIZE} ${PROJECT_NAME}.elf
175 )
```

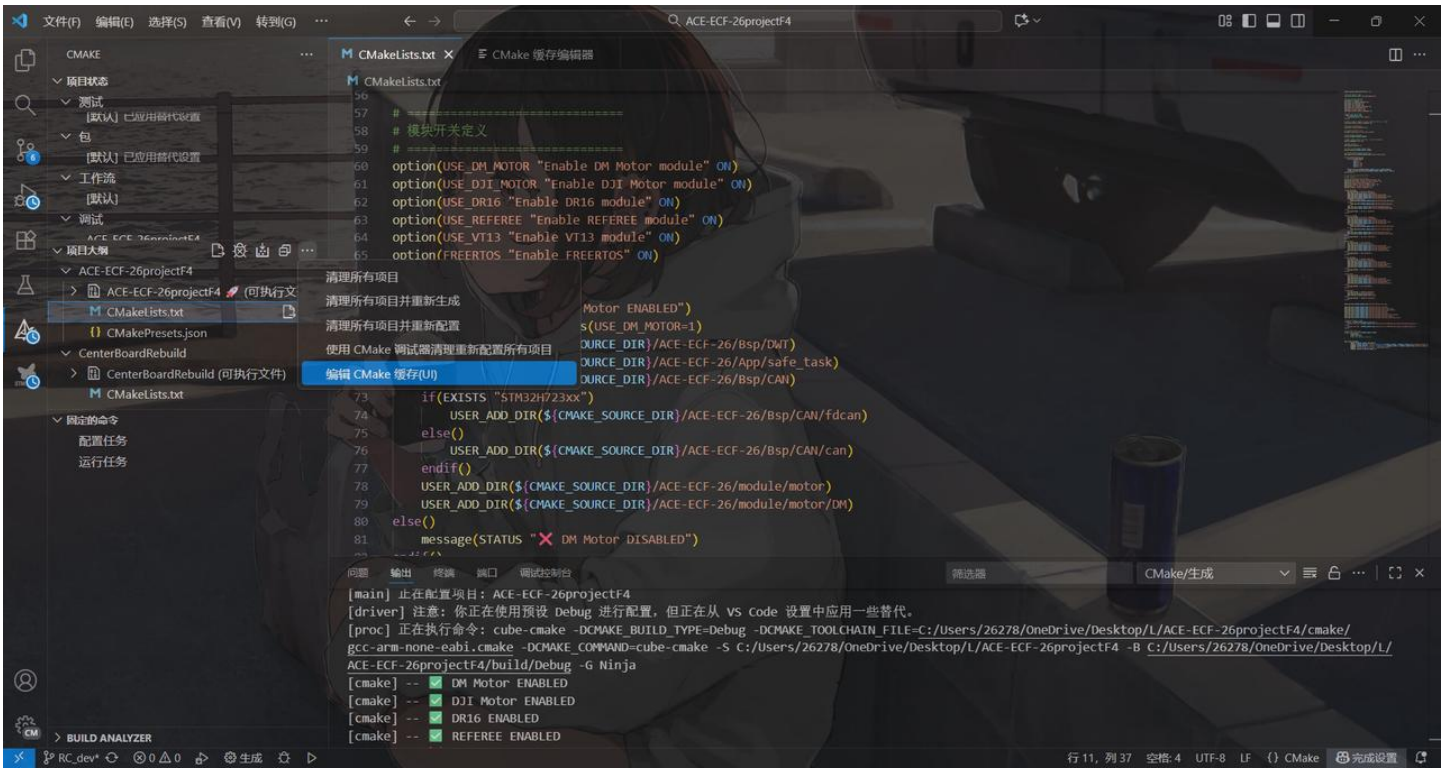
使用方法：

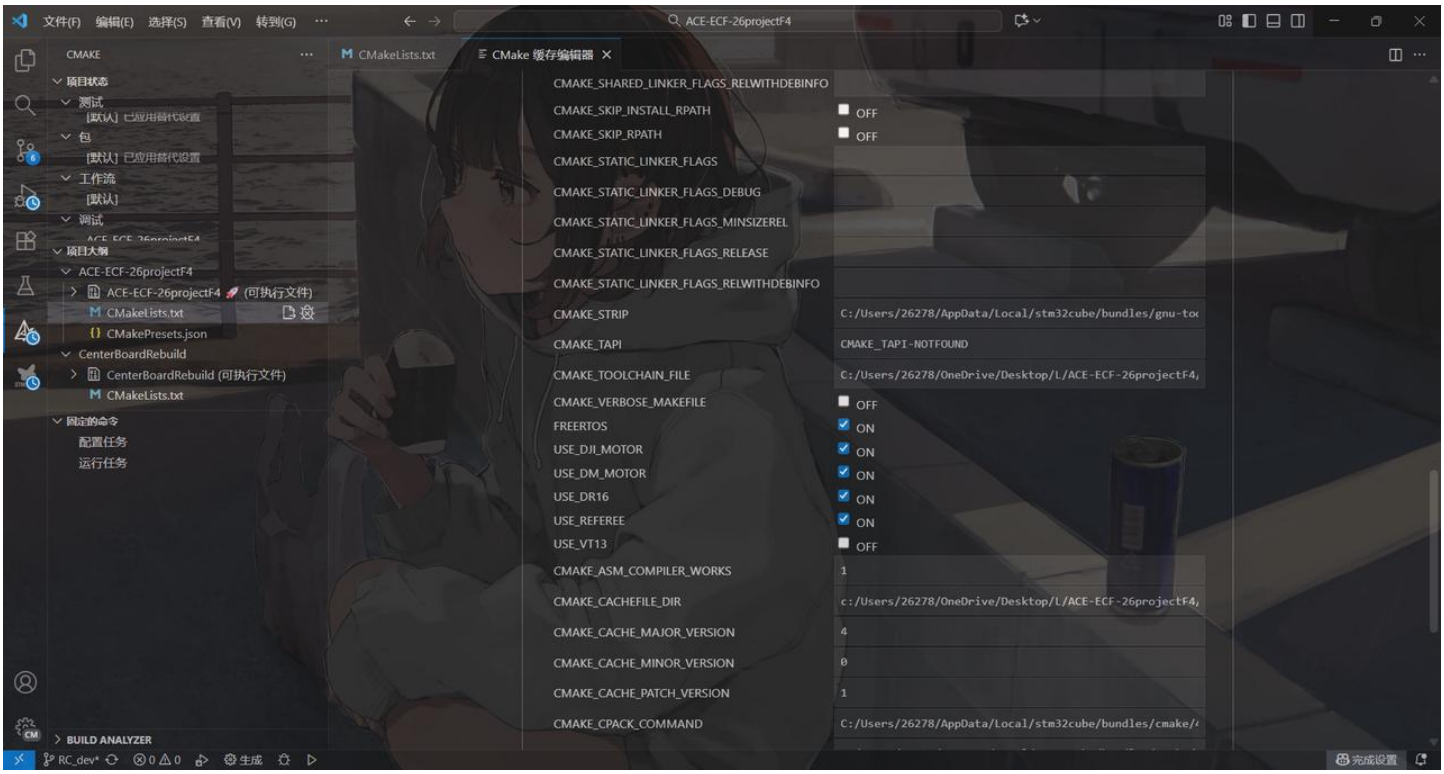
可视化开关选择启动模块

clion的开关位置如下；

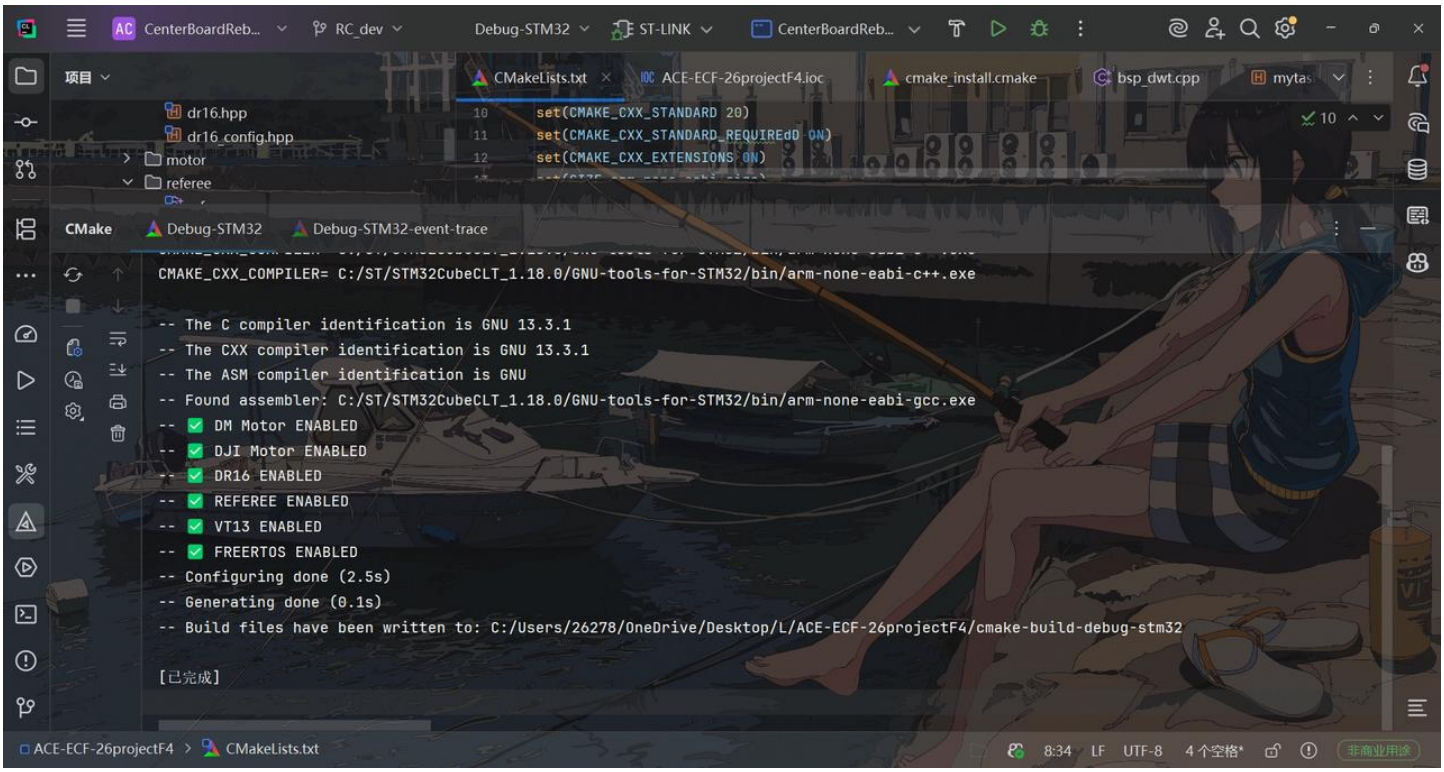


vscode的开关位置如下；


option(USE_DM_MOTOR "Enable DM Motor module" ON)
option(USE_DJI_MOTOR "Enable DJI Motor module" ON)
option(USE_DR16 "Enable DR16 module" ON)
option(USE_REFEREE "Enable REFEREE module" ON)
option(USE_VT13 "Enable VT13 module" ON)
option(FREERTOS "Enable FREERTOS" ON)
[main] 正在配置项目: ACE-ECF-26projectF4
[driver] 注意: 你正在使用预设 Debug 进行配置，但正在从 VS Code 设置中应用一些替代。
[cmake] -- DM Motor ENABLED
[cmake] -- DJI Motor ENABLED
[cmake] -- DR16 ENABLED
[cmake] -- REFEREE ENABLED


CMAKE_SHARED_LINKER_FLAGS_RELWITHDEBINFO
CMAKE_SKIP_INSTALL_RPATH
CMAKE_SKIP_RPATH
CMAKE_STATIC_LINKER_FLAGS
CMAKE_STATIC_LINKER_FLAGS_DEBUG
CMAKE_STATIC_LINKER_FLAGS_MINSIZEREL
CMAKE_STATIC_LINKER_FLAGS_RELEASE
CMAKE_STATIC_LINKER_FLAGS_RELWITHDEBINFO
CMAKE_STRIP
CMAKE_TAPI
CMAKE_TOOLCHAIN_FILE
CMAKE_VERBOSE_MAKEFILE
FREERTOS
USE_DJI_MOTOR
USE_DM_MOTOR
USE_DR16
USE_REFEREE
USE_VT13
CMAKE_ASM_COMPILER_WORKS
CMAKE_CACHEFILE_DIR
CMAKE_CACHE_MAJOR_VERSION
CMAKE_CACHE_MINOR_VERSION
CMAKE_CACHE_PATCH_VERSION
CMAKE_CPACK_COMMAND

cmake输出；



如何添加新的模块（框架撰写人）：

函数ADD_MODULE

```
ADD_MODULE(DM_MOTOR
${CMAKE_SOURCE_DIR}/ACE-ECF-26/Bsp/DWT
${CMAKE_SOURCE_DIR}/ACE-ECF-26/App/safe_task
${CMAKE_SOURCE_DIR}/ACE-ECF-26/Bsp/CAN
${CMAKE_SOURCE_DIR}/ACE-ECF-26/Bsp/CAN/fdcan
${CMAKE_SOURCE_DIR}/ACE-ECF-26/module/motor
${CMAKE_SOURCE_DIR}/ACE-ECF-26/module/motor/DM)
```

这样即可，函数已经封装好了

原理：

`option()` 创建缓存变量，`option(USE_${name} ...)` 会在 **CMakeCache.txt** 中生成一个布尔变量保存用户配置（ON/OFF），即使重新运行 CMake，也能记住上次的值

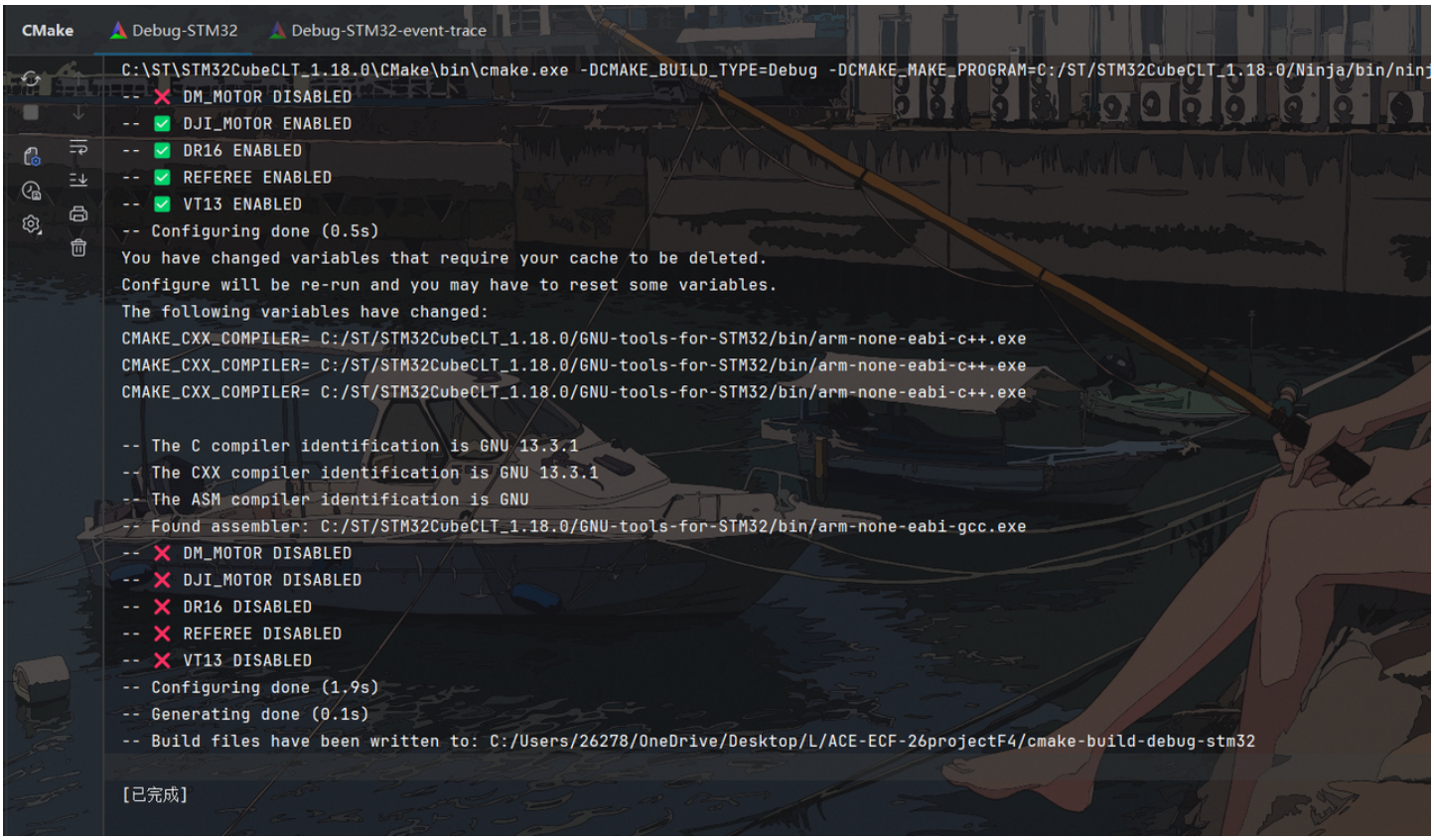
条件控制 `if(USE_${name})`，判断缓存变量，添加宏定义

`-D` 是在命令行传参时（就是我们所见的可视化配置）给 CMake 设置缓存变量

clion会出现的问题：

主要clion的cmake会出问题，vscode不会，下面会说原因

问题现象：我们的可视化配置并不改变option设置的布尔初始值，如下图的编译结果（你不打开还发现不了上面的内容）



解决方法：

1. 项目根目录->cmake文件夹->gcc-arm-none-eabi.cmake文件打开，修改

`set(CMAKE_CXX_COMPILER ${TOOLCHAIN_PREFIX}g++)` 这行的g++变成c++即可

2. 或者修改工具链里面的c++编译器改成g++也可

原因：你配置的工具链和生成的gcc-arm-none-eabi.cmake文件里面的工具链不一样

clion互博，先说vscode，vscode 的 CMake Tools 插件调用命令行CMake时参数由.cmake文件定义，比如 `cmake -DCMAKE_TOOLCHAIN_FILE=xxx.cmake` （`include("cmake/gcc-arm-none-eabi.cmake")`）

clion不一样，clion提前强制配置工具链中 `"你设置的路径"` ，然后再往后进行

`-DCMAKE_C_COMPILER="你设置的路径"`

`-DCMAKE_CXX_COMPILER="你设置的路径"`

然后自动生成的gcc-arm-none-eabi.cmake文件里面 `CMAKE_CXX_COMPILER` = ... `g++` ，而你配置的工具链是.....c++,于是 CMake 检测到 “欸？这两个值不一样，我要重新配置！” ,然后清空cache，触发所有 `option()` 重置成默认值

过往的故事——

@author yancy

代码块

```
cmake_minimum_required(VERSION 3.22)

# Set the project name
set(CMAKE_PROJECT_NAME CenterBoardRebuild)

# Setup compiler settings
set(CMAKE_C_STANDARD 11)
set(CMAKE_C_STANDARD_REQUIRED ON)
set(CMAKE_C_EXTENSIONS ON)
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIREdD ON)
set(CMAKE_CXX_EXTENSIONS ON)
set(SIZE arm-none-eabi-size)
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# Define the build type
if(NOT CMAKE_BUILD_TYPE)
    set(CMAKE_BUILD_TYPE "Debug")
endif()

# Enable compile command to ease indexing with e.g. clangd
set(CMAKE_EXPORT_COMPILE_COMMANDS ON)

# Add color diagnostics
set(CMAKE_COLOR_DIAGNOSTICS ON)

# Include toolchain file
include("cmake/gcc-arm-none-eabi.cmake")

# Core project settings
project(${CMAKE_PROJECT_NAME})

# Enable CMake support for ASM, C and CXX languages
enable_language(C CXX ASM)

# Remove warning
add_compile_options(-w)

# Create an executable object type
add_executable(${CMAKE_PROJECT_NAME})

# Add STM32CubeMX generated sources
add_subdirectory(${CMAKE_SOURCE_DIR}/cmake/stm32cubemx)

function(USER_ADD_DIR dir)
    file(GLOB USER_SOURCE
            ${dir}/*.h
```

```
            ${dir}/*.c
            ${dir}/*.hpp
            ${dir}/*.cpp
            ${dir}/*.cc
        )
        target_sources(${CMAKE_PROJECT_NAME} PUBLIC ${USER_SOURCE})
        target_include_directories(${CMAKE_PROJECT_NAME} PUBLIC ${dir})
    endfunction()



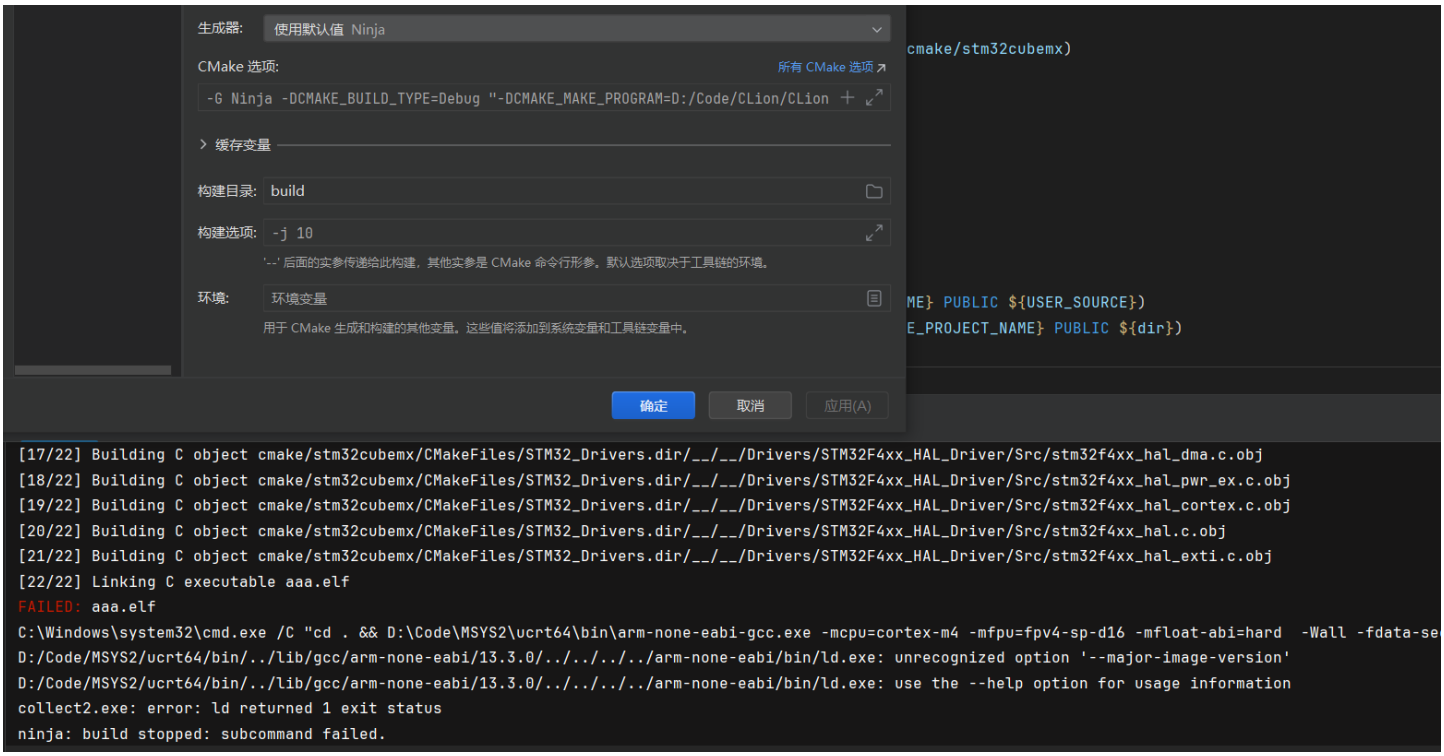
    USER_ADD_DIR(${CMAKE_SOURCE_DIR}/Middlewares/Third_Party/ARM_CMSIS/Include)
    USER_ADD_DIR(${CMAKE_SOURCE_DIR}/user/APP/Robo_ARM)

    target_link_libraries(${CMAKE_PROJECT_NAME}
            stm32cubemx
            # ${CMAKE_SOURCE_DIR}/Middlewares/ST/ARM/DSP/Lib/libarm_cortexM7lfdp_math.a
    )

    add_custom_command(
            TARGET ${PROJECT_NAME} POST_BUILD
            COMMAND ${CMAKE_OBJCOPY} -Oihex $<TARGET_FILE:${PROJECT_NAME}> ${PROJECT_BINARY_DIR}/${PROJECT_NAME}.hex
            COMMAND ${CMAKE_OBJCOPY} -Obinary $<TARGET_FILE:${PROJECT_NAME}> ${PROJECT_BINARY_DIR}/${PROJECT_NAME}.bin
            COMMENT "Building hex & bin file...\nEXCUTABLE SIZE:"
            COMMAND ${SIZE} ${PROJECT_NAME}.elf
    )
```

问题补充：

修改project name后，若构建出现如下Ninja生成报错，或是CMakeList错误根本没有构建选项



将生成器 `使用默认值` 修改为 任何一项 Makefile 生成器 后重新构建，再换回 默认 既可正常构建

如果以上改动都没有解决试试接下来的步骤：

首先阐述一下此问题的现象

新版本 6.15.0 中的 CMake 错误

转到解决方案



莫科兹
助理二级

2025-07-09 9：54 下午

今天，在使用 CubeMX 产品时，我升级到了 6.15.0 版本（之前是 6.14.1）。**升级后，生成项目代码时的 CMake 步骤失败**。即使在生成空项目时，CMake 仍然没有成功。我还尝试了以前成功刷写的以前工作项目，但它们在 CMake 步骤中也失败了。

显示以下错误消息：

```
CMake Error at D:/Program Files/JetBrains/CLion/bin/cmake/win/x64/share/cmake-3.31/Modules/CMakeTestCCompiler.cma
  The C compiler

    "C:/Program Files/GNU-Arm-Toolchain-eabi/bin/arm-none-eabi-gcc.exe"

  is not able to compile a simple test program.

  It fails with the following output:

    Change Dir: 'D:/project/keysking/P10_Clock/clock/cmake-build-debug/CMakeFiles/CMakeScratch/TryCompile-q0iun1'

    Run Build Command(s): "D:/Program Files/JetBrains/CLion/bin/ninja/win/x64/ninja.exe" -v cmTC_47519
    [1/2] C:\PROGRA~1\GNU-AR~1\bin\AR19DD~1.EXE   -std=gnu11 -fdiagnostics-color=always -o CMakeFiles/cmTC_47519.
    [2/2] C:\WINDOWS\system32\cmd.exe /C "cd . && C:\PROGRA~1\GNU-AR~1\bin\AR19DD~1.EXE   CMakeFiles/cmTC_47519.d
    FAILED: cmTC_47519.exe
    C:\WINDOWS\system32\cmd.exe /C "cd . && C:\PROGRA~1\GNU-AR~1\bin\AR19DD~1.EXE   CMakeFiles/cmTC_47519.dir/tes
    C:/PROGRA~1/GNU-AR~1/bin/../lib/gcc/arm-none-eabi/14.3.1/../../../../arm-none-eabi/bin/ld.exe: unrecognized o
    C:/PROGRA~1/GNU-AR~1/bin/../lib/gcc/arm-none-eabi/14.3.1/../../../../arm-none-eabi/bin/ld.exe: use the --help
    collect2.exe: error: ld returned 1 exit status
    ninja: build stopped: subcommand failed.

  CMake will not be able to correctly generate this project.
Call Stack (most recent call first):
  CMakeLists.txt:28 (project)
```

文件 .ioc，如下所示。

之后是解决方案

相关内容
CubeMX
的 MCU，
在STM32C
10-24
LL_USAR
ATOMIC
在STM32C
10-24
STM32Cu
务器插件
在 STM32
Studio Co
21
绑定
doStepS
败：找不
CMSIS_F
在 STM32
Studio Co
20
STM32 V
法在远程
SSH）
在 STM32
Studio Co
19

戈弗兰·格苏里
意法半导体员工
回应 Mocozzz

2025-07-17 上午8：58 - 已编辑
2025-07-18 上午3：34

你好@Mocozzz

该问题的根本原因与以前的 MX 版本中工具链文件的指定方式有关。以前，工具链文件在两个位置定义：CMakePresets.cmake 和 CMakeLists.txt。

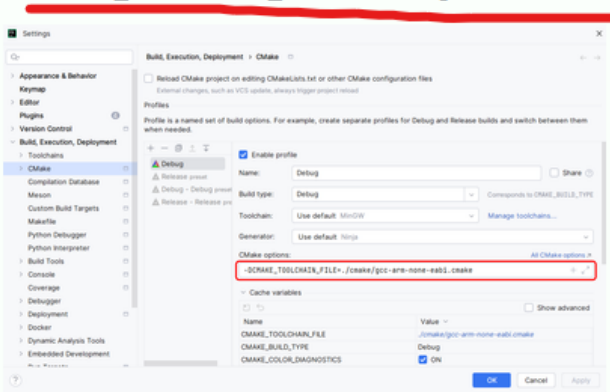
从 6.15 版开始，CMakeLists.txt 中的规范已被删除，现在仅保留在 CMakePresets.cmake 中。进行此更改是为了消除冗余并简化 GCC 和 starm-clang 之间的切换。

为了能够使用 Clion 构建 6.15 项目，我们需要在 Clion 项目中指定工具链文件：

- 转到菜单文件 -> 设置 -> 构建、执行、部署 -> CMake

- 在“CMake options”字段中，我们需要添加 GCC 的工具链文件变量：

-DCMAKE_TOOLCHAIN_FILE=./cmake/gcc-arm-none-eabi.cmake



注意：要使用 starm-clng 而不是 gcc，我们需要添加： -DCMAKE_TOOLCHAIN_FILE=./cmake/starm-clang.cmake

感谢

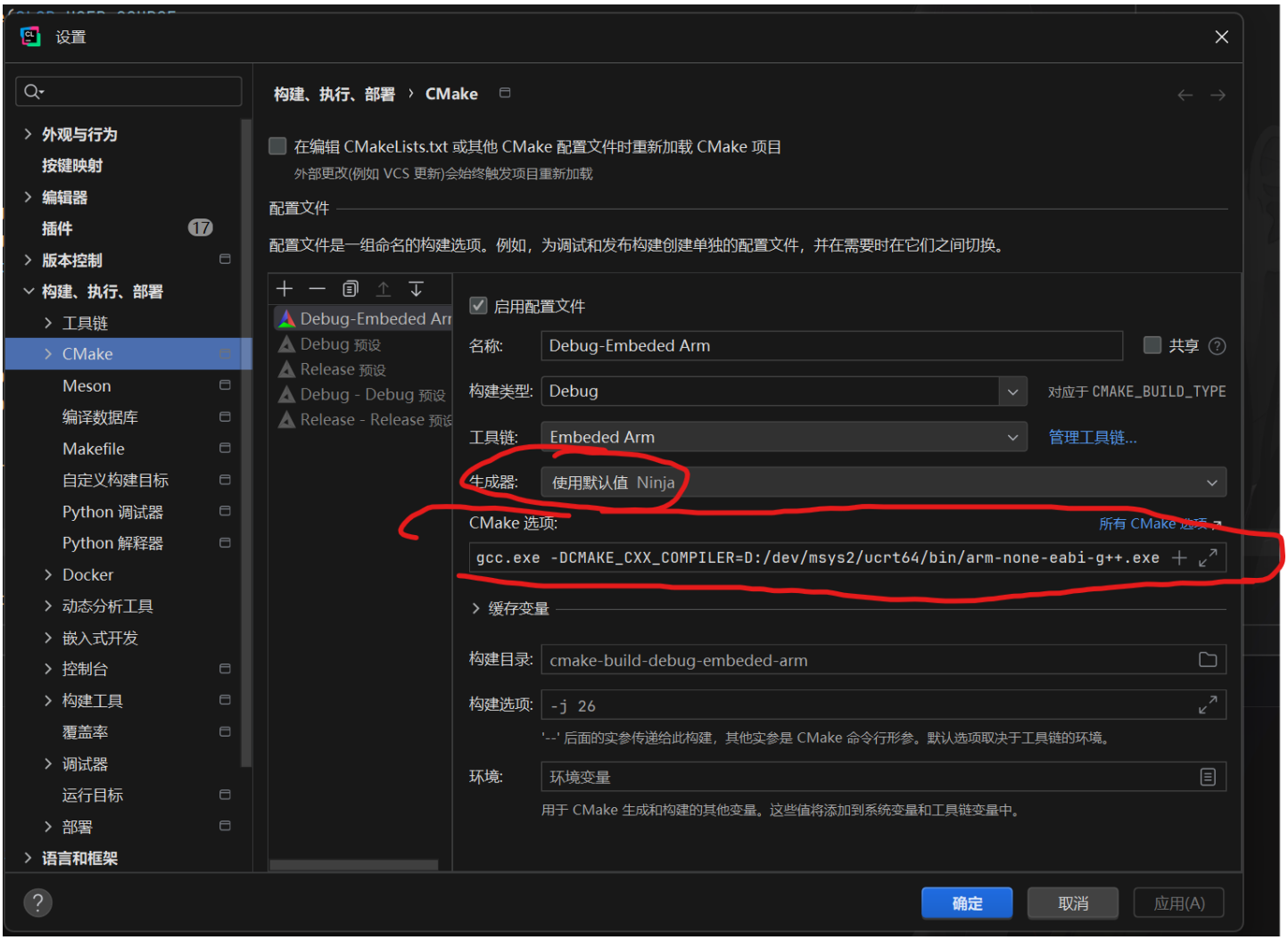
戈弗兰

为了更好地了解已回答的主题，请在解决您的问题或回答您的问题的回复上单击***“接受为解决方案”***。
在原始帖子中查看解决方案

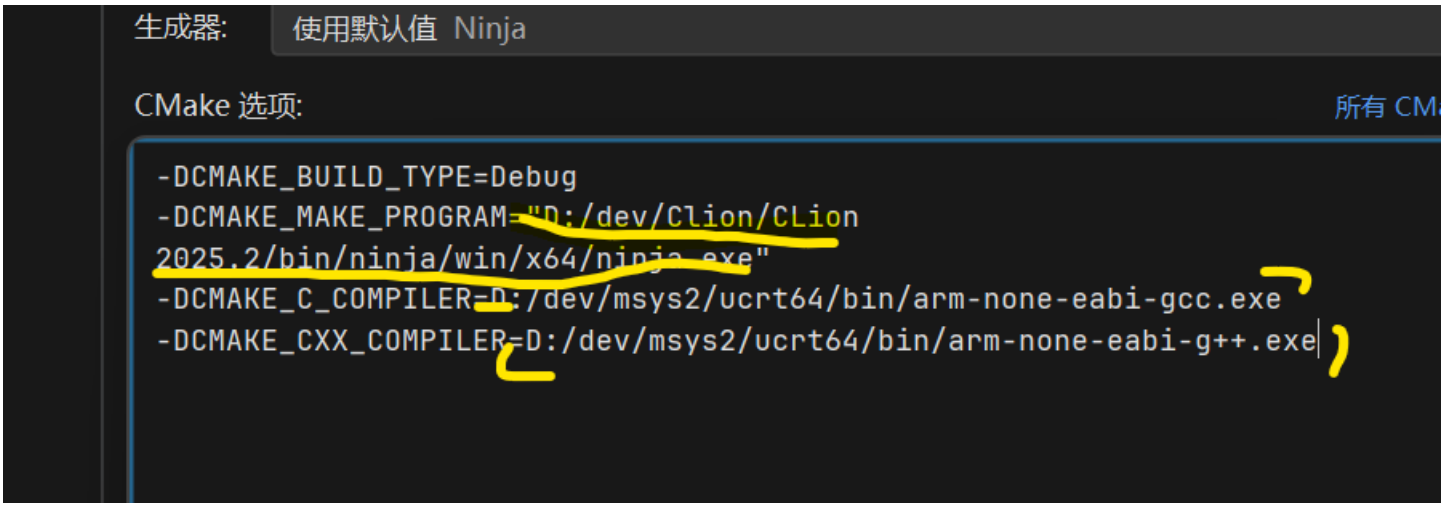
2 荣誉

答

简单来说就是编译器找不到你的GCC了，你需要告诉它（嘿！在这里呢！你个白痴）



首先使用默认值Ninja如果你不行说明给它指错位置了



分别是你电脑上的ninja地址，gcc地址，g++地址，自己找吧